Serialización y Deserialización de objetos

2.1. Qué es serializar / deserializar

Serializar = convertir un **objeto** en **bytes** para guardarlo (fichero, BBDD, caché) o enviarlo por red.

Deserializar = reconstruir el objeto desde esos bytes.

Se usa con:

- Persistencia rápida de objetos (formato binario).
- Comunicación entre procesos.
- Cachés/colas (cuando no usamos JSON/XML/protocolos).

En Java:

- **ObjectOutputStream** → escribe objetos (writeObject).
- **ObjectInputStream** → lee objetos (readObject).
- La clase a serializar debe **implementar java.io.Serializable** (interfaz marcador)

Términos clave:

- La clase debe implements Serializable
- Todos los **atributos** deben ser serializables.
- Si un atributo no lo es (p. ej., un Socket), márcalo como **transient**.
- **transient** = no se guarda en la secuencia de bytes → al deserializar, queda con su **valor por defecto** (null/0/false…).

2.2. Serializar un objeto a fichero

A continuación, se muestra un ejemplo para serializar objetos Persona

Clase Persona.java que debe implementar Serializable:

```
package model;

import java.io.Serializable;

public class Persona implements Serializable {
    // Recomendado para control de versión de clases (ver 6.1)
    private static final long serialVersionUID = 1L;

    private String nombre;
    private int    edad;
    private transient String password; // No se serializa

    public Persona(String nombre, int edad, String password) {
        this.nombre = nombre;
        this.edad = edad;
        this.password = password;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años) pwd=" + password;
    }
}
```

Clase DemoSerializar un objeto Persona:

```
package serializar_deserializar;

import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

import model.Persona;

public class DemoSerializarUno {
    public static void main(String[] args) {
        Persona p = new Persona("Ana", 25, "secreto123");

        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("persona.dat"))) {
            oos.writeObject(p); // Serializa 'p'
            System.out.println("OK: Objeto escrito en persona.dat");
        } catch (IOException e) {
            System.err.println("Error escribiendo: " + e.getMessage());
        }
    }
}
```

En Java, todo lo que entra o sale de un fichero/red pasa por flujos (streams).

- FileOutputStream: abre un fichero en disco para escribir bytes.
- ObjectOutputStream: "envuelve" al anterior para convertir objetos en bytes

```
FileOutputStream fos = new FileOutputStream("persona.dat");
ObjectOutputStream oos = new ObjectOutputStream(fos);
```

El método **writeObject(obj)** convierte internamente el objeto (Persona) en una secuencia de bytes y la envía al fichero.

Resumen **pasos**:

- **Marcar la clase con Serializable.**
- **Abrir un flujo de salida de objetos** (ObjectOutputStream).
- **Llamar a writeObject()** para convertir el objeto a bytes y guardarlo.
- **Cerrar el flujo** para no perder datos.

2.3. Deserializar un objeto a fichero

Ahora haremos el paso invertido.

```
package serializar_deserializar;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

import model.Persona;

public class DemoDeserializarUno {
    public static void main(String[] args) {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("persona.dat"))) {
            Persona p = (Persona) ois.readObject();
            System.out.println("Leído: " + p); // password será null
(transient)
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Error leyendo: " + e.getMessage());
        }
    }
}
```

Necesitamos un fichero (ej. persona.dat) que previamente se haya generado con ObjectOutputStream.writeObject().
En este fichero está la secuencia de bytes que representa al objeto.

Igual que al escribir usamos ObjectOutputStream, al leer usamos ObjectInputStream.

- FileInputStream → abre el fichero para leer bytes.
- ObjectInputStream → convierte esos bytes en objetos Java.

El método **readObject() reconstruye el objeto** desde los bytes del fichero.
Como devuelve Object, hay que hacer un **cast** al tipo original (Persona).

Resumen pasos:

- **Abrir flujo de entrada de objetos** (ObjectInputStream).
- **Llamar a readObject()** para reconstruir el objeto desde el fichero.
- **Hacer cast** al tipo correcto.
- **Cerrar el flujo**.
- **Trabajar con el objeto deserializado** como si lo acabáramos de crear en memoria.

2.4 Serializar/deserializar varios objetos

Cuando sea más de un objeto (será lo más probable) temenos dos opciones:

- Opción A. Escribir objetos **uno detrás de otro** (EOFException)
- Opción B. Serializar una **colección entera** (ej. ArrayList<Persona>)

Característica	**Opción A: Objetos sueltos (EOF)**	**Opción B: Colección entera (ArrayList)**
Escritura	Uno por uno con writeObject()	Una sola llamada con writeObject(lista)
Lectura	Bucle con EOFException	Una sola llamada y recorrer la lista
Claridad del código	Menos limpia	Más clara y mantenible
Añadir nuevos objetos	Fácil con append (sin cabecera extra)	Requiere reescribir toda la colección
Cargar grandes volúmenes	Mejor (puedes procesar uno a uno)	Malo (cargas todo en memoria)
Casos de uso	Logs, streaming, escritura incremental	Persistencia de listas completas

Ejemplo Opción A:

```
package serializar_deserializar;
import java.io.*;
import java.util.List;
import model.Persona;

/* OPCIÓN A: escribirlos seguidos y leer hasta EOF */

public class DemoSerializarDeserializarVariosEOF {
    public static void main(String[] args) {
        List<Persona> personas = List.of(
                new Persona("Ana", 25, "a"),
                new Persona("Luis", 31, "b"),
                new Persona("Marta", 28, "c")
        );

        // 1) Escribir varias personas seguidas
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("personas.dat"))) {
            for (Persona p : personas) {
                oos.writeObject(p);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 2) Leer hasta EOFException
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("personas.dat"))) {
            while (true) {
                Persona p = (Persona) ois.readObject();
                System.out.println("Leída: " + p);
            }
        } catch (EOFException eof) {
            System.out.println("Fin de fichero.");
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo Opción B:

```
package serializar_deserializar;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import model.Persona;

/* OPCIÓN B: escribir y leer una colección entera */

public class DemoSerializarDeserializarVariosColeccion {
    public static void main(String[] args) {
        List<Persona> personas = new ArrayList<>();
        personas.add(new Persona("Ana", 25, "a"));
```

```
        personas.add(new Persona("Luis", 31, "b"));
        personas.add(new Persona("Marta", 28, "c"));

        // 1) Escribir la colección completa de una sola vez
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("personas_lista.dat"))) {
            oos.writeObject(personas);
            System.out.println("Colección serializada en
personas_lista.dat");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 2) Leer la colección completa
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("personas_lista.dat"))) {
            List<Persona> leidas = (List<Persona>) ois.readObject();
            System.out.println("Colección recuperada:");
            for (Persona p : leidas) {
                System.out.println(" - " + p);
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Comparamos:

Aspecto	**Opción A: Objetos uno a uno (EOF)**	**Opción B: Colección completa (ArrayList)**
Fichero de salida	personas.dat	personas_lista.dat
Escritura	Bucle que recorre la lista y hace oos.writeObject(p) por cada objeto.	Un solo oos.writeObject(personas) para guardar toda la colección de golpe.
Lectura	while(true) con ois.readObject(), termina capturando EOFException.	Un único readObject(), se castea a List<Persona> y luego se recorre con for-each.
Control de flujo	Depende de **excepción** (EOFException) para saber cuándo parar de leer.	No depende de excepciones: sabe cuántos objetos hay porque la lista se recupera entera.
Ventajas	Se pueden añadir objetos incrementalmente (con append). Útil para procesar objetos de uno en uno (streaming).	Código más claro y sencillo. Recuperas la lista entera lista para usar.
Desventajas	Lectura menos limpia (uso de EOFException). Difícil acceder a un objeto concreto sin recorrer todos.	Necesita cargar toda la colección en memoria. Añadir objetos nuevos requiere reescribir todo.

Conclusión:

- Si quiero persistir un conjunto de objetos completo → uso colección (Opción B).
- Si quiero ir guardando objetos de forma incremental, como un log → uso objetos seguidos con EOF (Opción A).

2.5 Añadir objetos a un fichero existente (sin reescribir cabecera)

ObjectOutputStream escribe una **cabecera** al inicio.

Cuando creas un ObjectOutputStream nuevo, su **constructor escribe automáticamente al principio del flujo** una firma y versión del formato de serialización:

- **STREAM_MAGIC** = 0xAC ED
- **STREAM_VERSION** = 0x00 05

Esa cabecera sirve para que ObjectInputStream verifique: "esto es un flujo de objetos Java válido y empieza aquí".

Si abres un fichero **en modo append** y creas **otro** ObjectOutputStream, **ese segundo OOS vuelve a escribir la cabecera en mitad del fichero**.
Al leer, ObjectInputStream espera **el siguiente objeto**, pero se encuentra otra cabecera (AC ED 00 05) y lanza:

- StreamCorruptedException: invalid stream header … (o "unexpected block data").

Patrones seguros para añadir objetos

Patrón A — Mantener un único ObjectOutputStream abierto

- Abres el OOS una vez, vas llamando a writeObject() tantas veces como quieras, y lo cierras al final.
- **No escribes cabeceras adicionales** porque no recreas el OOS.
- Útil en procesos "vivos" (logs, colas). No sirve si vas a **reabrir** el programa mañana y seguir añadiendo.

Ejemplo:

```
package serializar_deserializar;

import model.Persona;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.util.List;

/* Patron A  Mantener un único ObjectOutputStream abierto */
public class AnadirObjetosPatronUnicoOOS {
    public static void main(String[] args) {
        List<Persona> personas = List.of(
                new Persona("Ana", 25, "a"),
                new Persona("Luis", 31, "b"),
                new Persona("Marta", 28, "c")
        );

        // Abrimos un único ObjectOutputStream
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("patronA.dat"))) {
            // Vamos escribiendo objetos uno a uno
            for (Persona p : personas) {
                oos.writeObject(p);  // cada llamada serializa un objeto
                System.out.println("Escrita persona: " + p);
            }
            // Al salir del try-with-resources se cierra automáticamente
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Notas:

- Aquí no hay "append" real: el fichero se escribe **todo en una sola ejecución**.
- Útil si en tu aplicación generas todos los objetos de golpe (ej. exportar lista de alumnos).
- No sirve si quieres **reabrir el programa otro día** para añadir más objetos → porque necesitarías un nuevo OOS (y ahí se reescribiría cabecera).

Patrón B — Reabrir el fichero en append sin reescribir cabecera

- Primera escritura: **OOS normal** (deja la cabecera al inicio del fichero).
- Siguientes escrituras: **OOS especial** que **omite** la cabecera.
- ¿Cómo? Subclasando ObjectOutputStream y sobreescribiendo writeStreamHeader().

Ejemplo: hay dos clases.

```
package serializar_deserializar;

import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;

/* Patrón B — Reabrir fichero en append sin cabecera extra */
// Clase especial que omite la cabecera en escrituras posteriores
public class AppendingObjectOutputStream extends ObjectOutputStream {
    public AppendingObjectOutputStream(OutputStream out) throws IOException {
        super(out);
    }

    @Override
    protected void writeStreamHeader() throws IOException {
        reset(); // evita escribir STREAM_MAGIC/STREAM_VERSION otra vez
    }
}
```

```
package serializar_deserializar;

import model.Persona;
import java.io.*;
import java.util.List;

/* Patron B Reabrir fichero en append sin cabecera extra*/
public class AnadirObjetosAppendSinCabecera {
    private static final String FICHERO = "patronB.dat";

    public static void main(String[] args) {
        // 1ª tanda: escribe cabecera + objetos
        escribir(List.of(
                new Persona("Ana", 25, "a"),
                new Persona("Luis", 31, "b")
        ));

        // 2ª tanda: añade más objetos SIN reescribir cabecera
        escribir(List.of(
                new Persona("Marta", 28, "c")
        ));
    }

    private static void escribir(List<Persona> personas) {
        File f = new File(FICHERO);
        boolean existe = f.exists() && f.length() > 0;
```

```
        try (OutputStream fos = new FileOutputStream(f, true);
             ObjectOutputStream oos = existe
                     ? new AppendingObjectOutputStream(fos) // sin cabecera
                     : new ObjectOutputStream(fos))          // con cabecera
        {
            for (Persona p : personas) {
                oos.writeObject(p);
                System.out.println("Escrita persona: " + p);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Notas:

- La **primera vez** que se escribe el fichero, se usa un ObjectOutputStream normal (deja cabecera).
- En ejecuciones posteriores, se detecta que el fichero ya existe y tiene datos, y se usa AppendingObjectOutputStream, que **omite la cabecera**.
- Así se pueden ir añadiendo objetos poco a poco (ej. programa que se ejecuta cada día y añade registros).
- Al leer con ObjectInputStream, todo se interpreta bien porque solo hay una cabecera al inicio.

Comparativa de ambas opciones:

Patrón A	**Patrón B**
Se abre un solo ObjectOutputStream.	Se puede reabrir el fichero en sesiones distintas.
No hay riesgo de cabecera extra.	Se usa un ObjectOutputStream especial sin cabecera.
Solo sirve si escribes todos los objetos en una ejecución.	Sirve para añadir objetos en varias ejecuciones diferentes.

2.6 Concepto de serialVersionUID

Cuando serializas un objeto, Java guarda **la versión de la clase** junto con los datos.
Esa "versión" es un número llamado **serialVersionUID**.

- Es como un **DNI de la clase**.
- Sirve para que, cuando vuelvas a deserializar, Java pueda comprobar que la **clase en el código coincide con la clase que se usó al escribir el fichero**.

¿Qué pasa si no lo pongo?

Si no declaras un serialVersionUID, Java genera uno automáticamente a partir de la estructura de la clase (atributos, métodos, etc.).

Problema: si más adelante cambias la clase (añades un atributo, renombras uno, cambias modificadores...), Java calculará un **nuevo UID** automáticamente. Cuando intentes deserializar un fichero creado con la clase antigua, Java detectará que los UID no coinciden y lanzará: java.io.InvalidClassException: model.Persona; local class incompatible

Si declaras el serialVersionUID en tu clase, **controlas la compatibilidad**.

```
package model;

import java.io.Serializable;

/* Controla serialVersionUID */
public class Personav2 implements Serializable {
    private static final long serialVersionUID = 1L;

    private String nombre;
    private int edad;
    private transient String password;

    public Personav2(String nombre, int edad, String password) {
        this.nombre = nombre;
        this.edad = edad;
        this.password = password;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + ")";
    }
}
```

- Ahora, aunque cambies la clase (añadir un campo nuevo, por ejemplo), **si mantienes el mismo serialVersionUID**, Java permitirá leer los objetos guardados con la versión anterior.

- Los campos nuevos simplemente se inicializan a valores por defecto (null, 0, false).

En **resumen**:

- Con serialVersionUID fijo → controlas la compatibilidad.
- Sin él → dependes de la generación automática de Java, y cualquier cambio rompe la lectura.

2.7 Buenas prácticas

- **Siempre usar try-with-resources** → cierra flujos automáticamente.
- **Usar colecciones enteras** (opción B) cuando los objetos se gestionan en bloque.
- **Opción A (EOF)** solo cuando sea necesario un "log" incremental.
- **Evitar serializar objetos muy grandes o dependientes de red** (ej. Sockets). Para interoperabilidad mejor JSON, XML, etc.